Day 2 - MLPs & Backpropagation

From one neuron to many layers - and how a network finds its gradients

Annika Stiehl · Deep Learning · M.Sc. · SRH

Today

Time	
09:00-10:30	Recap of day 1 - with pictures and a browser tool: slope, chain rule • Activation functions • A hidden layer solves XOR
10:45-12:15	MLPs: layers, parameters, softmax • Backpropagation: idea, worked example, PyTorch autograd
13:00-14:30	Lab 2: run → change → explain (warm-up, backprop step by step, your first MLP in PyTorch)
14:45-16:00	Lab 2: PyTorch MLPs on FashionMNIST and EEG features • Q&A

💻 Today we also code during the lecture: open `day2_code_breaks.ipynb` from the course repository in Colab (CPU is enough).

Recap: how does a neuron learn?

Day 1 in 30 minutes - with a browser tool, pictures and gears

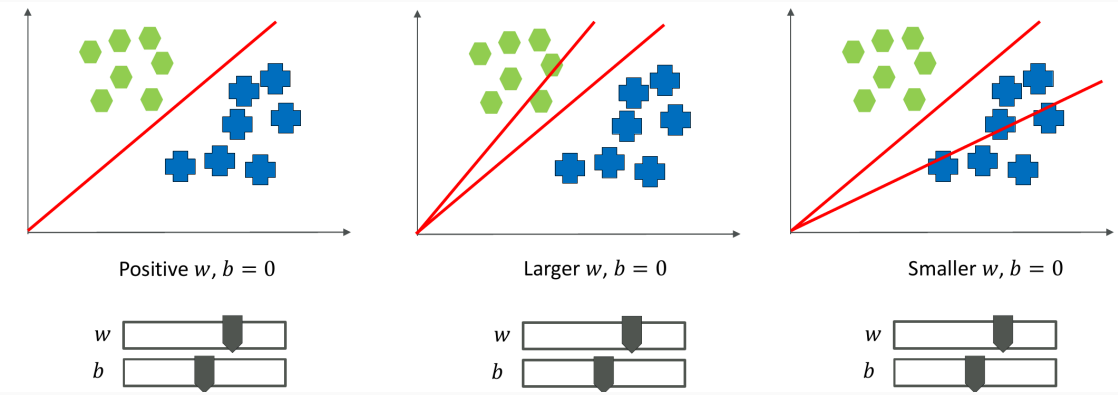
Try it: train your dragon

jalammar.github.io/visual-interactive-guide-basics-neural-networks

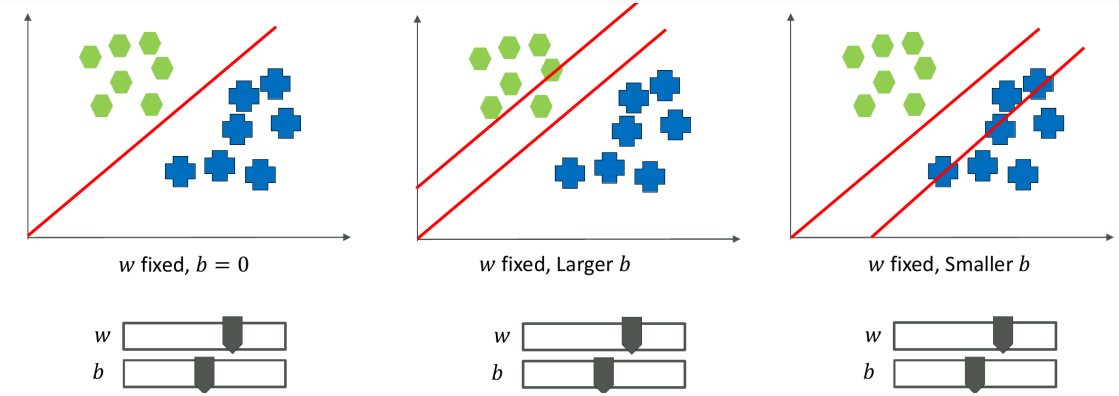
House prices from the area - one neuron: price = weight × area + bias

1. **Train Your Dragon:** turn the dials weight and bias yourself. Can you get the error below 799?
2. Which dial changes the error more?
3. **Automation:** press the gradient-descent step buttons. What happens to the error, the weight and the bias?
4. **And Then There Were Two:** a second input (bathrooms). Try to optimise this by hand.

One dial at a time: partial derivatives



turn **only** w: the line rotates

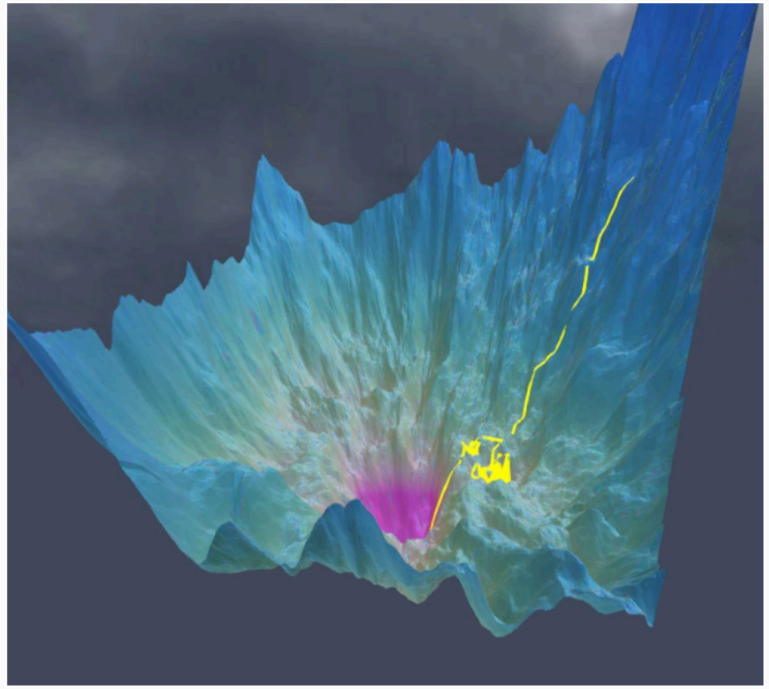


turn **only** b: the line shifts

- how fast does the loss change if I turn one dial a little? → $\partial L/\partial w$ and $\partial L/\partial b$ (partial derivatives)
- **gradient** = all these slopes together: $\nabla L = (\partial L/\partial w,\ \partial L/\partial b)$
- exactly what you felt in the dragon tool: each dial has its own slope

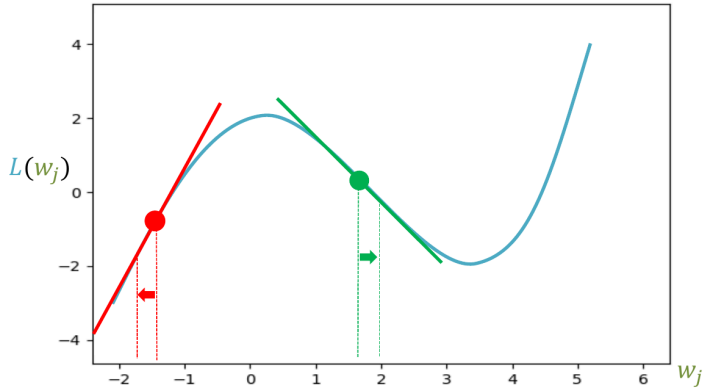
Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

Gradient descent: roll downhill, guided by the slope



Determine the *rate of change* of our loss function L with respect to a parameter's change

1. „Increasing w_j a little bit will *increase* the loss a little bit" → so decrease w_j!
2. „Increasing w_j a little bit will *decrease* the loss a little bit" → so increase w_j!



Formally:

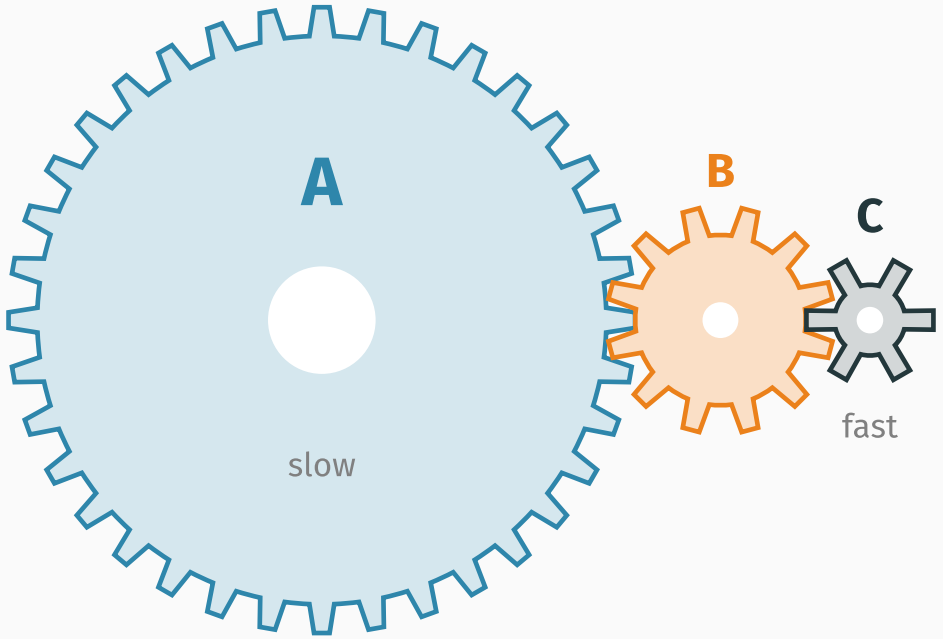
find $\frac{\partial L}{\partial w_j}$ and update $w_j \leftarrow w_j - \alpha \frac{\partial L}{\partial w_j}$

α is called the „learning rate"

- the algorithm cannot see the landscape - it only feels the **slope** under its feet
- slope positive (red): going right goes uphill → decrease w. Slope negative (green) → increase w
- update rule from day 1: $w \leftarrow w - \eta\, \partial L / \partial w$ (learning rate η, in the figure called α)

Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

The chain rule: gears



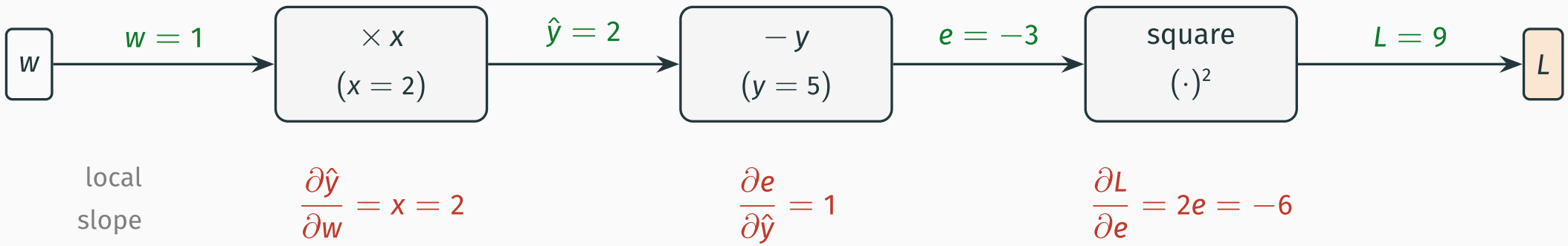
B turns $3\times$ as fast as A $\quad \frac{dB}{dA} = 3$

C turns $2\times$ as fast as B $\quad \frac{dC}{dB} = 2$

C turns $3 \cdot 2 = 6\times$ as fast as A $\quad \frac{dC}{dA} = \frac{dC}{dB} \cdot \frac{dB}{dA} = 6$

- each gear only knows how fast it turns compared to its neighbour
- **chain rule:** last gear compared to the first = the product of the small ratios
- in a network: turn a weight a little → how much does the loss change? = the product of the small slopes along the way

The chain rule gives the MSE gradient



$$\frac{\partial L}{\partial w} = 2 \cdot 1 \cdot (-6) = -12$$

Multiply the small slopes: $\dfrac{\partial L}{\partial w} = 2e \cdot 1 \cdot x = -12.$

For n samples: the same for each sample, then the mean:

$$\frac{\partial L}{\partial w} = \frac{1}{n} \sum_{i=1}^{n} 2\, e_i\, x_i$$

💻 Code break 1: loss and gradient in code

```
x = torch.tensor([2.104, 1.600, 2.400])   # area in 1000 sq ft
y = torch.tensor([399.9, 329.9, 369.0])   # price in 1000 $
w, b, lr = 0.0, 0.0, 0.1
for step in range(20):
    e = w * x + b - y                     # error per house
    loss = (e ** 2).mean()                # MSE
    dw = (2 * e * x).mean()               # chain rule: 2 * error * input
    db = (2 * e).mean()
    w = w - lr * dw                       # gradient descent
    b = b - lr * db                       # gradient descent
    print(step, round(loss.item(), 1))
```

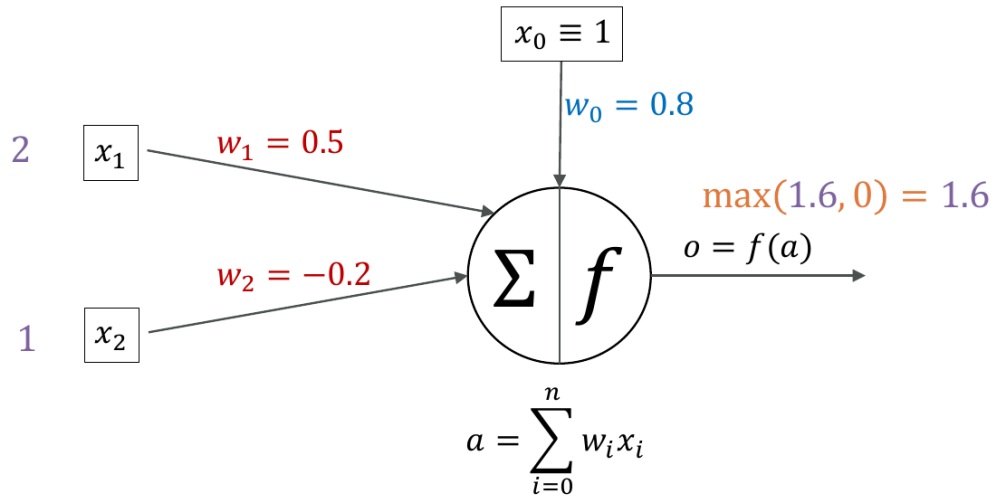
👥 Your turn (5 min) - day2_code_breaks.ipynb, code break 1

1. run the cells: plot the houses, train, plot the fitted line - does the loss go down?
2. change `lr = 0.1` to `lr = 0.2` and run again. What happens? And with `lr = 0.01`?

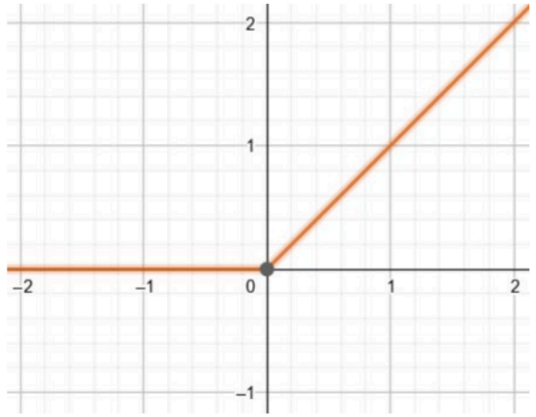
Activation functions

What a neuron does after the weighted sum

A neuron: weighted sum, then an activation function



$2 \cdot 0.5 + 1 \cdot (-0.2) + 0.8 = 1.6$



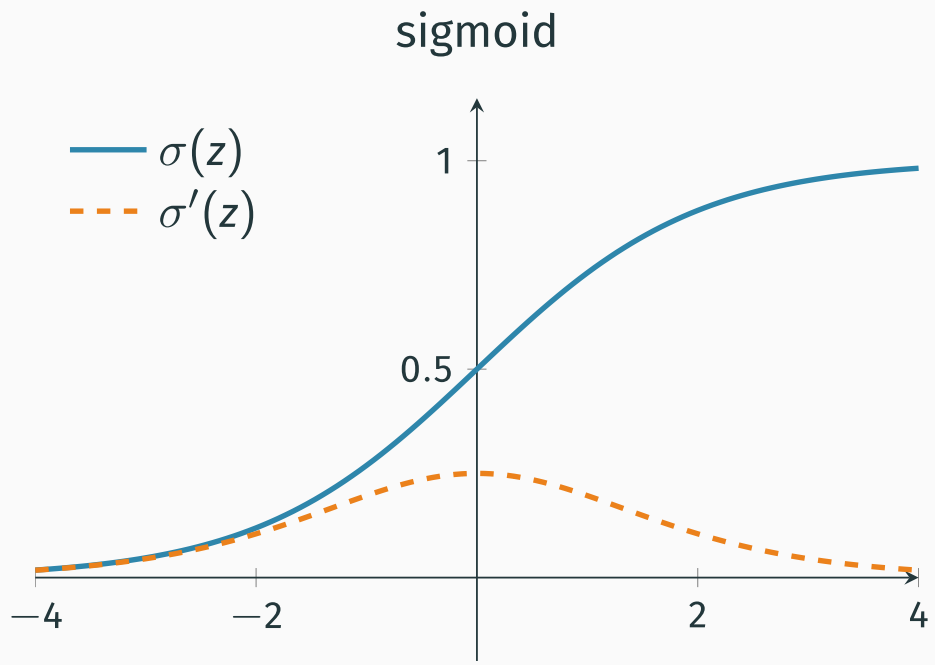
$f(z) = \max(z, 0)$

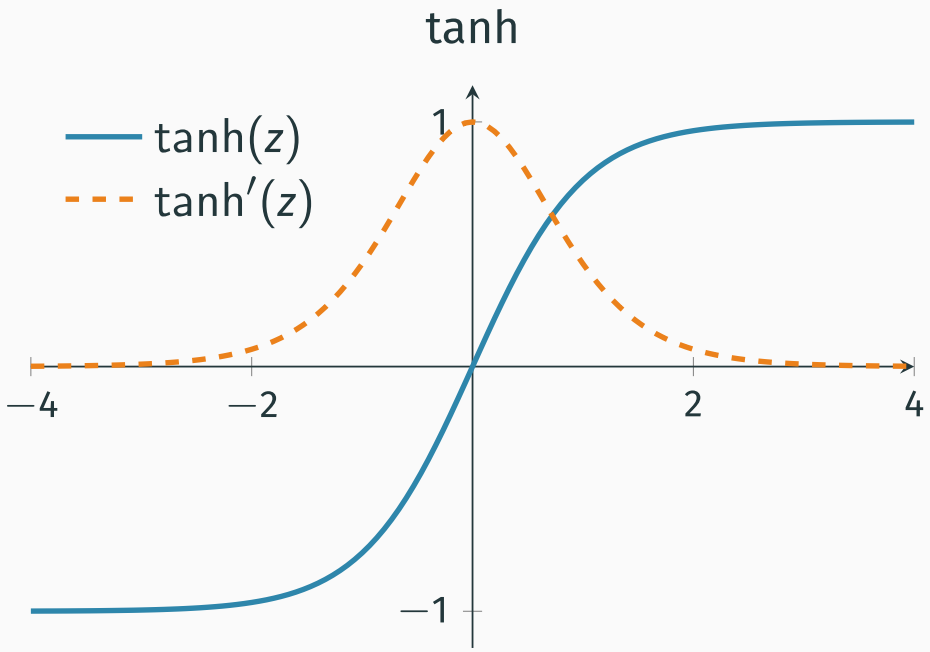
x_i: inputs
b: bias weight
a: activation
w_i: weights
$f(z)$: activation function
o: output

- the **activation function** f decides what the neuron passes on
- here **ReLU**: $f(z) = \max(z, 0)$ - negative sums become 0, positive sums pass unchanged
- day 1: the perceptron used a step function - it is flat everywhere (slope 0), so gradient descent cannot learn with it

Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

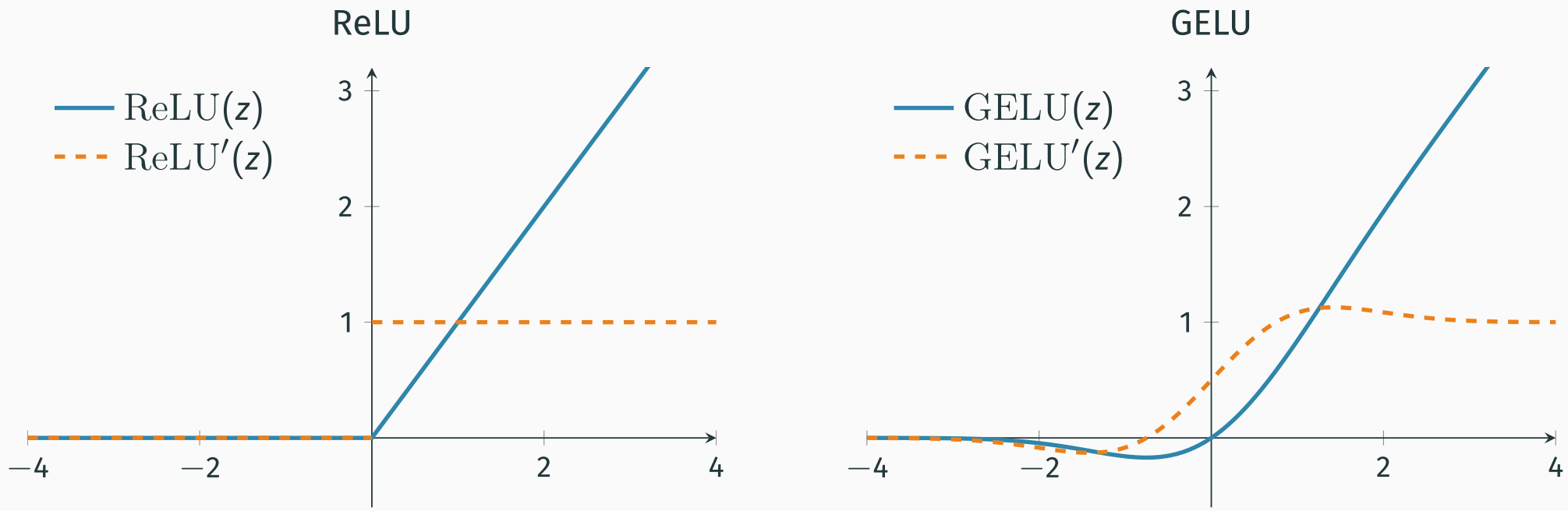
Activation functions: sigmoid and tanh





- $\sigma(z) = \dfrac{1}{1+e^{-z}}$ squeezes any number into $(0, 1)$ - like a probability
- $\tanh(z)$ squeezes into $(-1, 1)$
- Both get flat for large $|z|$ (saturation): slope ≈ 0 ⇒ almost no learning signal

Activation functions: ReLU and GELU



- $\mathrm{ReLU}(z) = \max(0, z)$: slope 1 for $z > 0$, 0 for $z < 0$ - cheap, no saturation for positive z
- GELU: a smooth ReLU, the default in Transformers (GPT, BERT)
- **Rule of thumb:** ReLU (or GELU) in the hidden layers

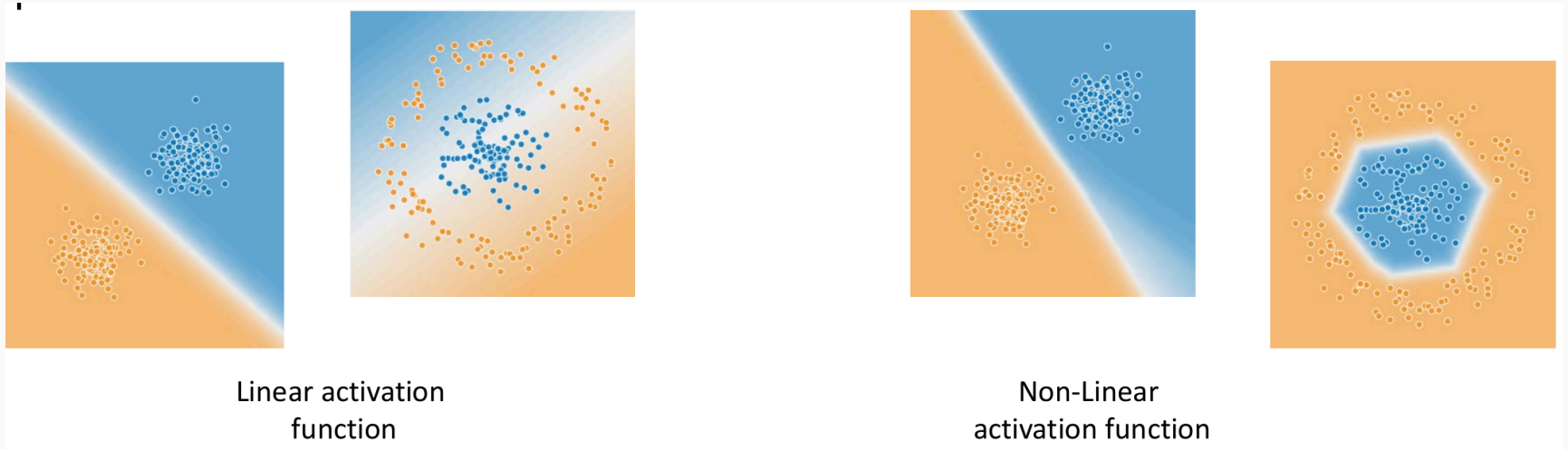
💻 Code break 2: activation functions in PyTorch

```
z = torch.tensor([-3.0, -1.6, 0.0, 1.6, 3.0])
print(torch.relu(z))
print(torch.sigmoid(z))
print(torch.tanh(z))
```

👥 Your turn (3 min) - code break 2

1. run it: which function turns negative numbers into 0? Which one gives numbers between 0 and 1?
2. put your own numbers into `z` (e.g. `-100.0` and `100.0`). Run the plot cell below it.

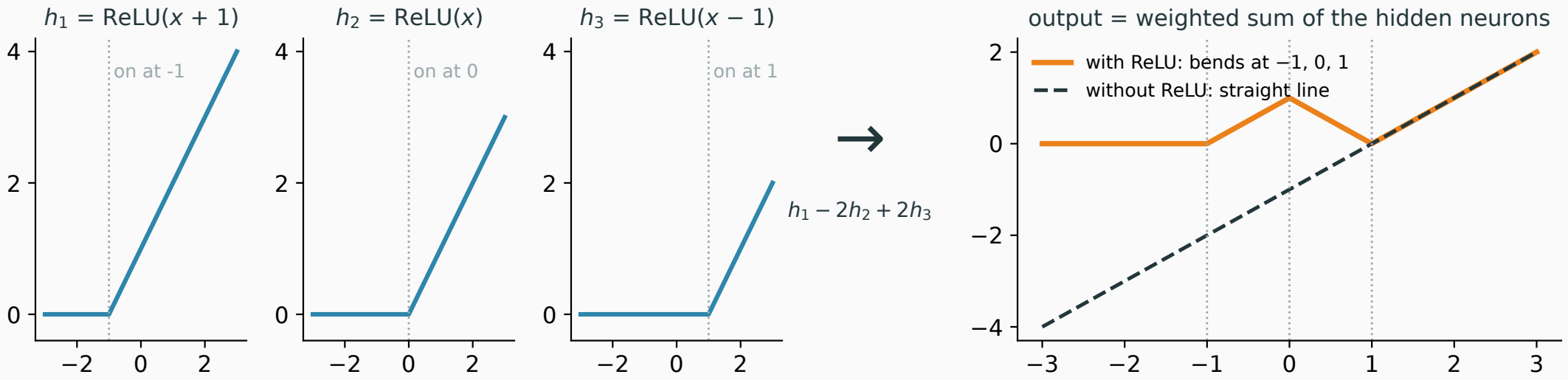
Why the activation must be non-linear



- without a non-linear activation, stacked layers collapse into one linear layer: only straight boundaries (left)
- with a non-linear activation (e.g. ReLU) the network can bend the boundary (right)

Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

How ReLU neurons bend a line



- each ReLU neuron is a **hinge**: 0 until its sum becomes positive, then a straight ramp
- the output adds the hinges ⇒ the line bends where a neuron switches on. Without ReLU: straight
- more neurons = more bends ⇒ a network can follow almost any curve; training chooses the bends

💻 Code break 3: without an activation, layers stay a straight line

```
x = torch.linspace(-3, 3, 100).reshape(-1, 1)        # 100 inputs, 1 feature
linear = nn.Sequential(nn.Linear(1, 8), nn.Linear(8, 1))
bent   = nn.Sequential(nn.Linear(1, 8), nn.ReLU(), nn.Linear(8, 1))
plt.plot(x, linear(x).detach(), label="no activation")
plt.plot(x, bent(x).detach(), label="with ReLU")
plt.legend(); plt.show()
```

👥 Your turn (4 min) - code break 3

1. run it: which line is straight, which one is bent?
2. add more `nn.Linear(8, 8)` layers to `linear` (without activation). Is it still straight?
3. replace `nn.ReLU()` by `nn.Tanh()`

Try it: activations in TensorFlow Playground

playground.tensorflow.org - dataset Circle, inputs X_1, X_2

1. 2 hidden layers with 4 neurons, activation **Linear**. Press play. What happens?
2. same network, activation **ReLU**. Now?
3. activation **Sigmoid**. Faster or slower than ReLU?
4. dataset Spiral, activation **ReLU**: find a small network that solves it

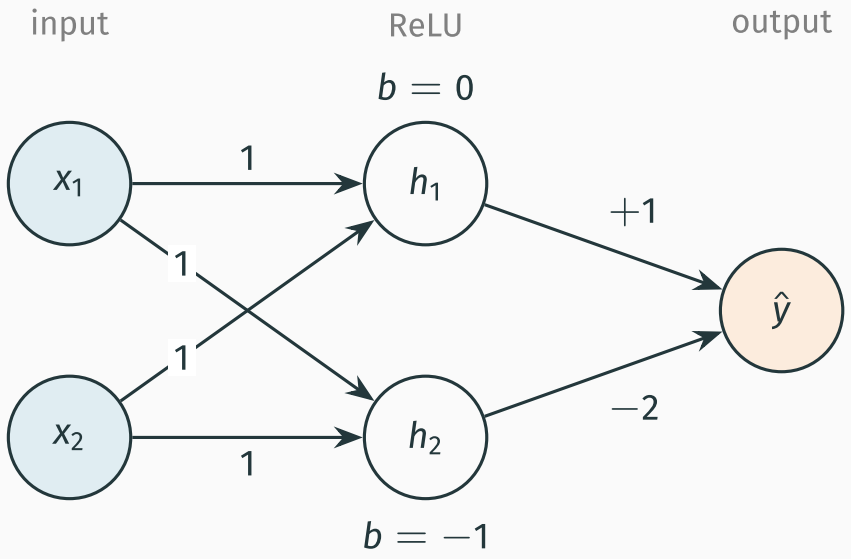
👥 Your turn (10 min)

In your browser, in pairs. Note the test loss after about 300 epochs.

Multilayer perceptrons

Many neurons, in layers

A hidden layer solves XOR



$h_1 = \mathrm{ReLU}(x_1 + x_2)$ - counts the ones

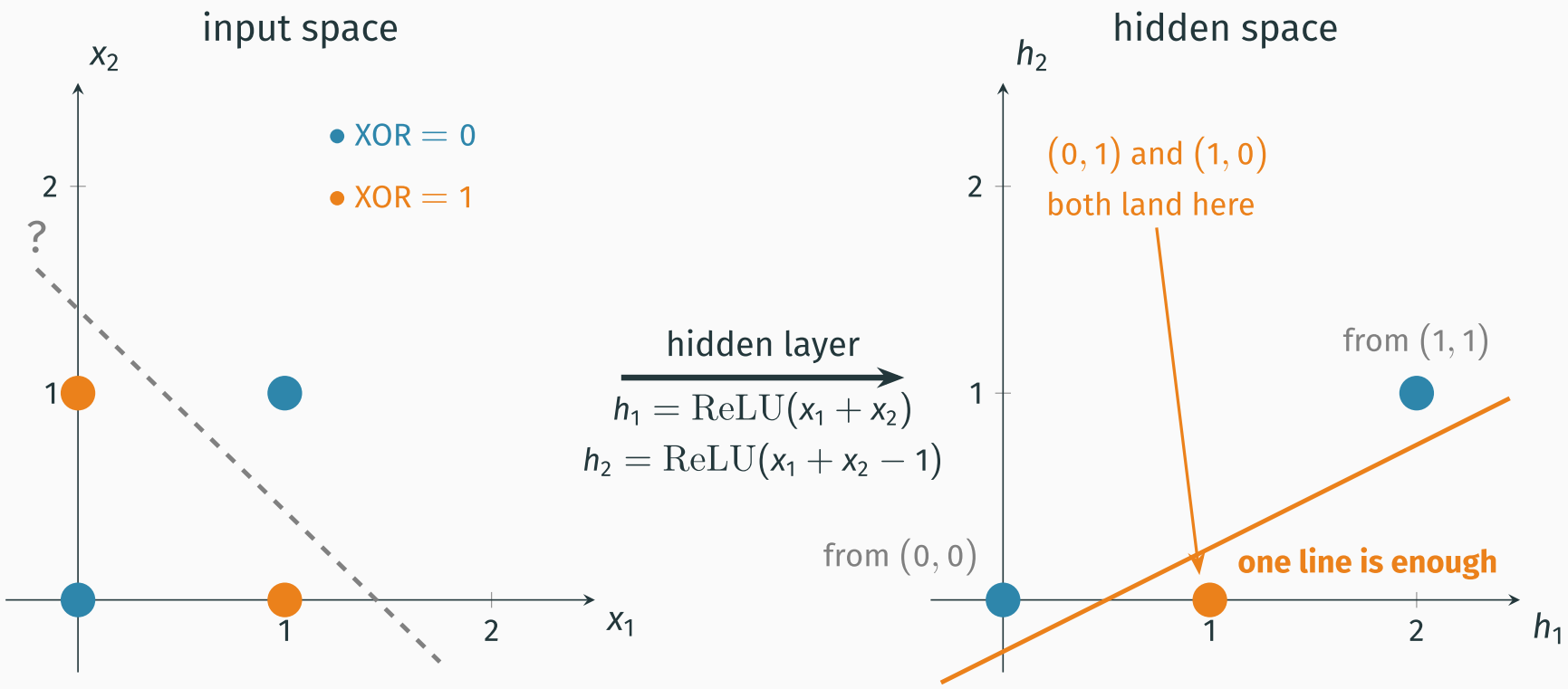
$h_2 = \mathrm{ReLU}(x_1 + x_2 - 1)$ - only "on" for $(1, 1)$

$\hat{y} = h_1 - 2\,h_2$

❓ Think about it

Try one point: $(1, 1)$. What are h_1, h_2 and $\hat{y}$? (Hint: $\mathrm{ReLU}(z) = \max(0, z)$, XOR of $(1, 1)$ is 0)

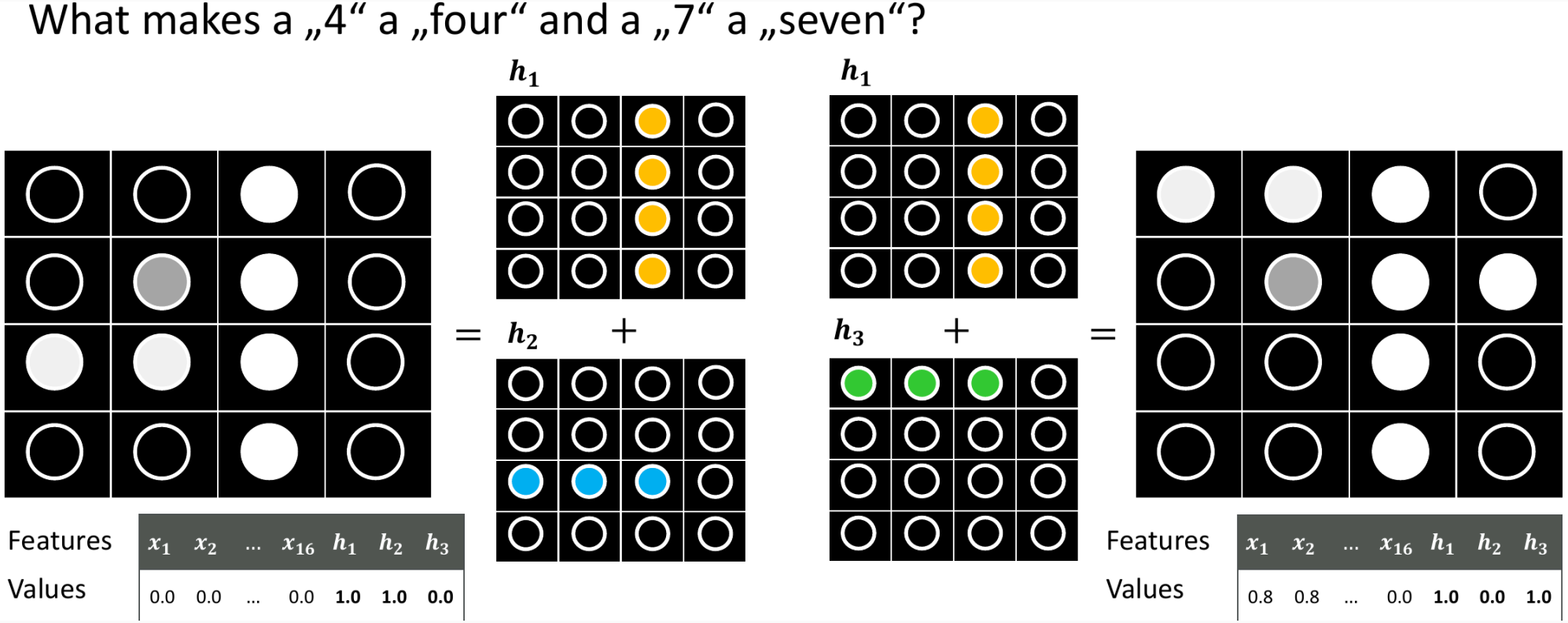
What the hidden layer did



🔑 Key takeaway

The hidden layer moves the points - in the new space one straight line is enough. Here we chose the weights by hand; **training finds such weights automatically**.

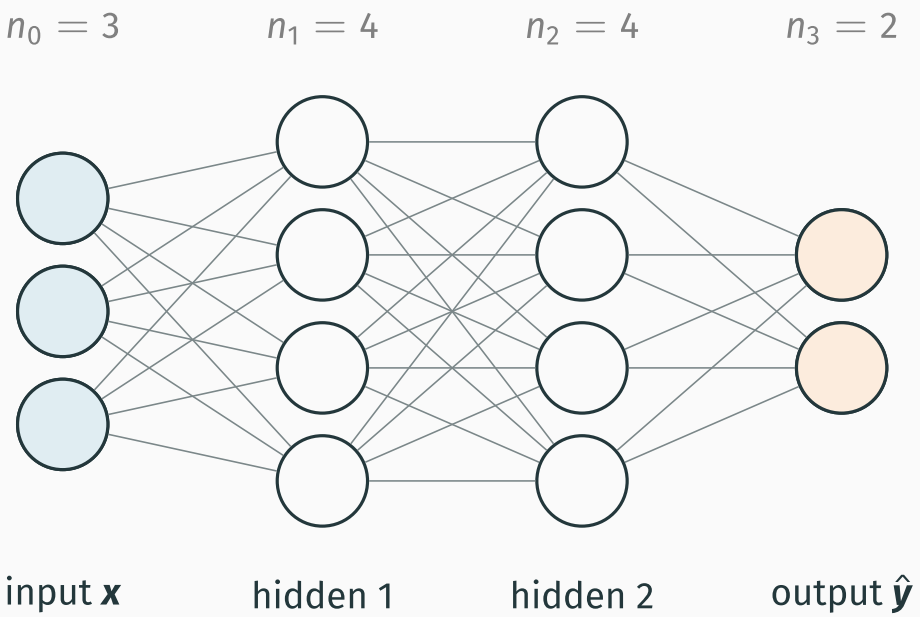
Hidden neurons detect parts



- each hidden neuron reacts to one part: h_1 = vertical stroke, h_2 = middle bar, h_3 = top bar
- "4" = h_1 + h_2, "7" = h_1 + h_3: the output layer only has to combine the parts
- in a real network nobody chooses these parts - training finds them

Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

A multilayer perceptron (MLP)



- **input layer:** the features $\boldsymbol{x}$ (here 3)
- **hidden layers:** neurons in between (here 2 layers with 4 neurons)
- **output layer:** the prediction (here 2 numbers)
- **width** = neurons per layer, **depth** = number of layers
- **dense / fully connected:** every neuron sees all neurons of the layer before
- **feed-forward:** information flows from left to right only

Every neuron: weighted sum + bias, then activation - like yesterday.

Counting parameters

Each layer: (inputs × outputs) weights + (outputs) biases

Example: 3 → 4 → 4 → 2 (previous slide)

layer	weights	biases	total
3 → 4	12	4	16
4 → 4	16	4	20
4 → 2	8	2	10
			46

Today's EEG model in the lab: 90 → 64 → 64 → 2

5 824 + 4 160 + 130 = **10 114** parameters - but only about 3 600 training windows

PyTorch counts for you:

```
sum(p.numel() for p in model.parameters())
```

💻 Code break 4: an MLP in PyTorch

```
model = nn.Sequential(
    nn.Linear(90, 64), nn.ReLU(),     # hidden layer 1
    nn.Linear(64, 64), nn.ReLU(),     # hidden layer 2
    nn.Linear(64, 2))                 # output: 2 classes
print(model)
print(sum(p.numel() for p in model.parameters()))   # number of parameters
x = torch.randn(8, 90)                # a batch: 8 windows, 90 features
print(model(x).shape)                 # 8 rows, 2 outputs
```

👥 Your turn (5 min) - code break 4

1. run it: how many parameters? What is the output shape?
2. predict first, then try: hidden width 128 instead of 64. How many parameters now?
3. add a third hidden layer

The output layer depends on the task

task	last layer gives	turn into	loss	PyTorch
regression ("how much?")	one number	nothing - the number is the answer	mean squared error	`nn.MSELoss()`
classification ("which one?")	one score per class (logits)	probabilities with softmax	cross-entropy	`nn.CrossEntropyLoss()`

- logits = the raw scores of the last layer, any numbers (also negative)
- the PyTorch loss applies the softmax itself - you give it the logits

🧠 EEG corner

Seizure vs. no seizure: 2 classes, so the last layer has 2 outputs + softmax.

Softmax and cross-entropy

Three classes, raw outputs (logits) $z = (2,\ 1,\ 0)$:

	cat	dog	horse
logit z_k	2	1	0
probability (softmax)	**0.67**	0.24	0.09

Softmax: all positive, sum = 1, the largest score gets the largest probability.

Cross-entropy = $-\log$(probability of the true class):

- true class cat: $-\log 0.67 = 0.41$ (small loss)
- true class horse: $-\log 0.09 = 2.41$ (large loss)

🔑 Key takeaway

Confident and right: small loss. Confident and wrong: large loss.

💻 Code break 5: softmax and cross-entropy

```
logits = torch.tensor([[2.0, 1.0, 0.0]])          # one sample, 3 classes
print(torch.softmax(logits, dim=1))               # probabilities
print(F.cross_entropy(logits, torch.tensor([0]))) # true class 0 (cat)
print(F.cross_entropy(logits, torch.tensor([2]))) # true class 2 (horse)
```

👥 Your turn (3 min) - code break 5

1. run it - the same numbers as on the slide before?
2. make the model more confident: `[[5.0, 1.0, 0.0]]`. What happens to the two losses?

Backpropagation

The chain rule, applied to the whole network

Why we need a clever way to get gradients

Gradient descent needs $\partial L / \partial w$ for **every** weight.

Idea 1: turn each knob (finite differences)
2 forward passes per weight
FashionMNIST MLP: 269 322 weights ⇒ ≈ 540 000 forward passes - for one step 😱

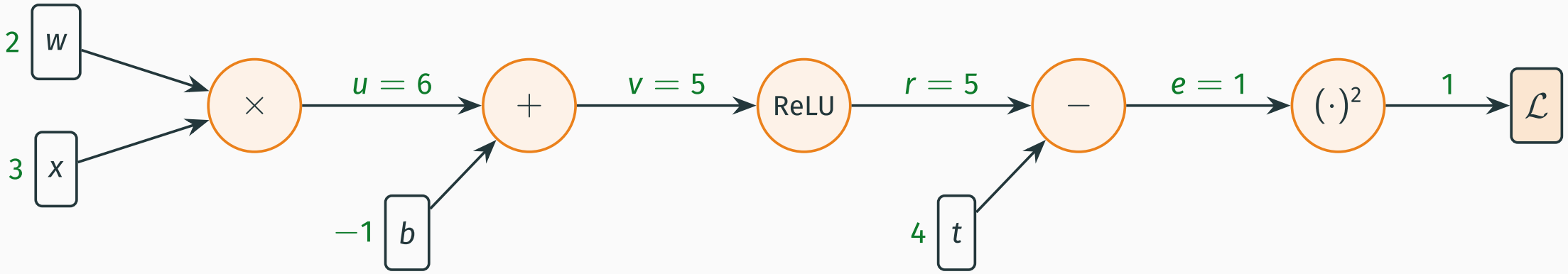
Idea 2: backpropagation
1 forward pass + 1 backward pass ⇒ all gradients at once

❓ Think about it

The loss depends on the first layer only through the later layers. Which slopes could the first layer re-use from the later layers?

Rumelhart, Hinton & Williams, Nature 1986

A network is a graph of small machines

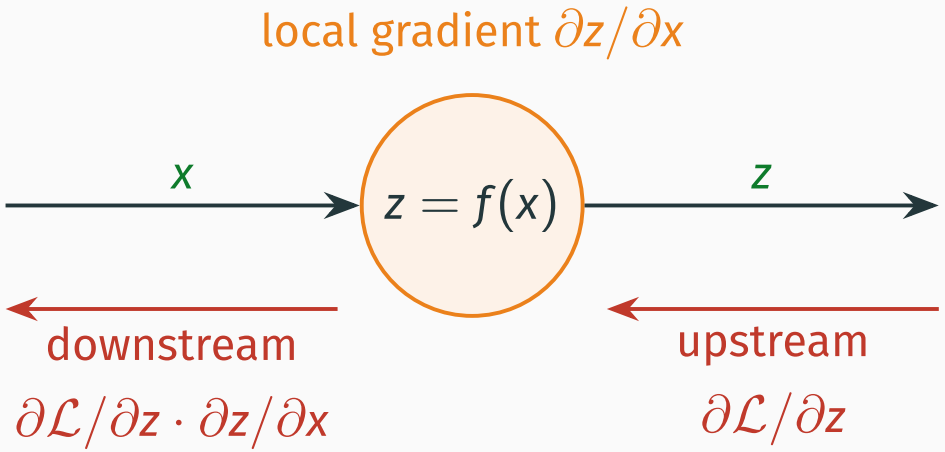


green: forward value

One neuron with ReLU, one sample: $L = (\mathrm{ReLU}(w\,x + b) - t)^2$ with $x = 3$, $w = 2$, $b = -1$, target $t = 4$

- **forward** (green): compute from left to right and store every value
- **backward**: walk from right to left and compute the slopes

Backward: upstream × local

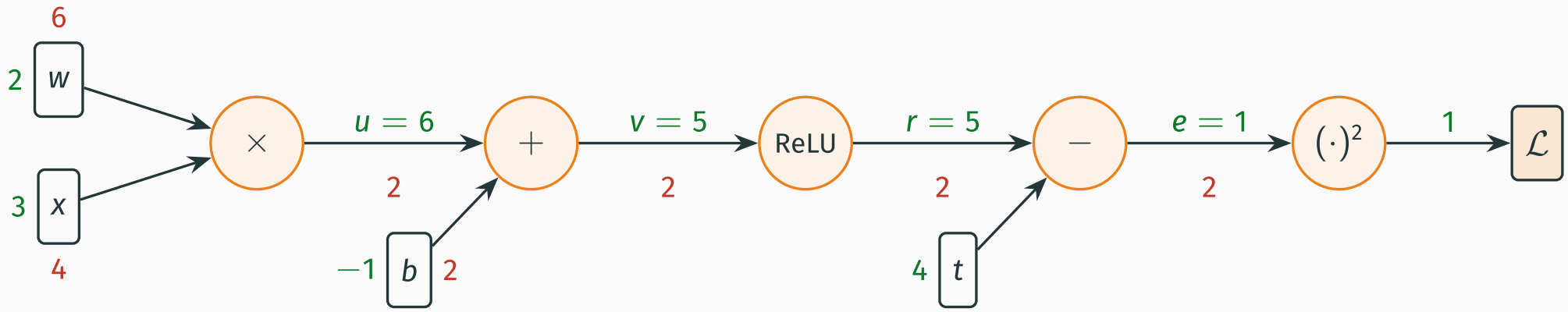


Every machine gets a slope from the right (upstream), multiplies it with its own slope (local) and passes the result to the left:

$$\text{downstream} = \text{upstream} \times \text{local}$$

- each machine only needs its own small rule
- that is the chain rule from the recap, one machine at a time

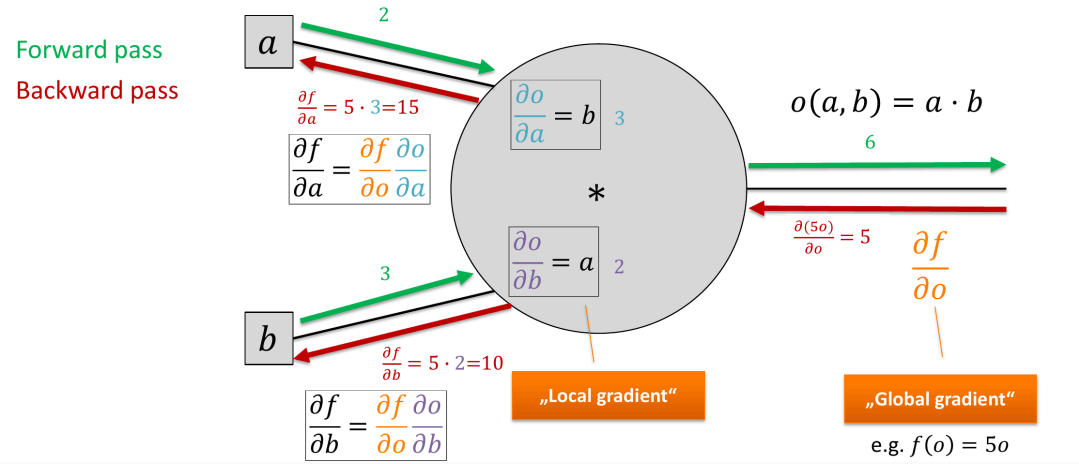
Worked example: backward step by step



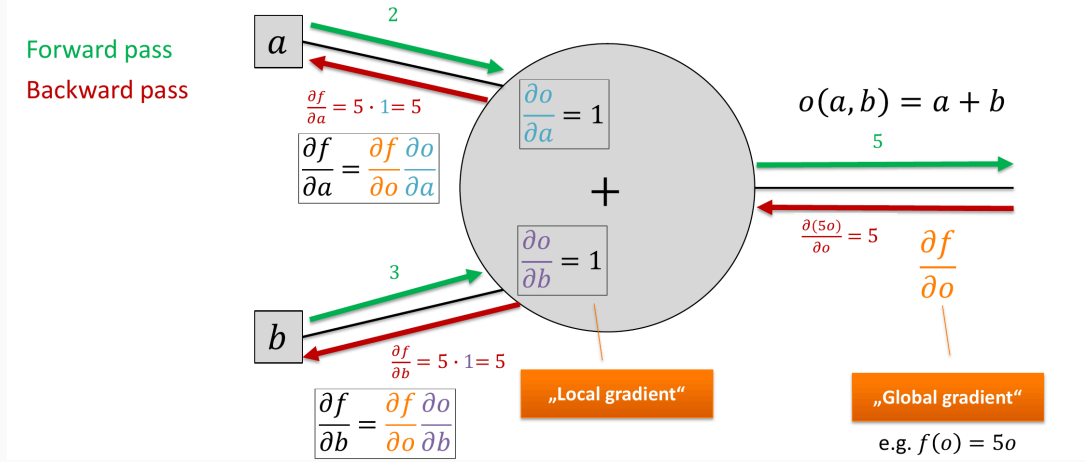
green: forward value red: gradient $\partial \mathcal{L}/\partial(\cdot)$

machine	local slope	upstream × local
$L = e^2$	$2e = 2$	$\partial L/\partial e = 2$
$e = r - t$	1	$\partial L/\partial r = 2 \cdot 1 = 2$
$r = \mathrm{ReLU}(v), v = 5 > 0$	1	$\partial L/\partial v = 2$
$v = u + b$	1 and 1	$\partial L/\partial u = 2,\ \partial L/\partial b = 2$
$u = w \cdot x$	$x = 3$ and $w = 2$	$\partial L/\partial w = 2 \cdot 3 = 6,\ \partial L/\partial x = 2 \cdot 2 = 4$

Patterns you will see again and again



multiply = "switcher": a gets upstream × b, b gets upstream × a



add = "distributor": both inputs get the upstream slope

- **ReLU** = "gate": passes the slope if its input was > 0, else 0 • a value **used twice**: the two slopes are added

❓ Think about it

In the worked example, change b from -1 to -7. What are $\partial L/\partial w$ and $\partial L/\partial b$ now? Why do we call this a dead ReLU?

Figure: Deep Learning Bootcamp (DLBC-I), A. Schiendorfer, TH Ingolstadt

Try it: watch backprop in your browser

A - Backprop Explainer (interactive)
xnought.github.io/backprop-explainer

- a small network learns a curvy function
- press play: the fit improves step by step
- follow one forward and one backward pass through the network

B - 3Blue1Brown: "What is backpropagation really doing?" (video, almost no formulas)
3blue1brown.com/lessons/backpropagation

👥 Your turn (10 min)

A in pairs - or B together on the projector. Look for: which numbers flow forward (left → right), which flow backward (right → left)?

🔑 Key takeaway

`loss.backward()` does exactly this - for millions of weights.

Bertucci & Kahng, VISxAI 2021 • 3Blue1Brown, Neural networks, chapter 3

💻 Code break 6: autograd does backprop for you

```
x, t = torch.tensor(3.0), torch.tensor(4.0)
w = torch.tensor(2.0, requires_grad=True)   # PyTorch: "I need the slope for w"
b = torch.tensor(-1.0, requires_grad=True)

L = (torch.relu(w * x + b) - t) ** 2        # forward: PyTorch records the graph
L.backward()                                # backward: upstream x local, automatically
print(L.item(), w.grad, b.grad)             # 1.0  tensor(6.)  tensor(2.)
```

👥 Your turn (4 min) - code break 6

1. run it: the same numbers as in the worked example (6 and 2)?
2. change `b` to `-7.0` (dead ReLU). 3. change `x` to `6.0`

Autograd in the training loop

```
opt = torch.optim.SGD(model.parameters(), lr=0.1)
for xb, yb in loader:
    opt.zero_grad()                          # 1. delete old slopes
    loss = F.cross_entropy(model(xb), yb)    # 2. forward: prediction + loss
    loss.backward()                          # 3. backward: all slopes
    opt.step()                               # 4. update: w = w - lr * slope
```

❓ Think about it

You forget `opt.zero_grad()`. Does the program crash? What happens? → code break 7

💻 Code break 7: why `zero_grad()`?

```
w = torch.tensor(2.0, requires_grad=True)
for i in range(3):
    L = (w * 3.0 - 4.0) ** 2
    L.backward()
    print(w.grad.item())          # the slopes add up!
```

👥 Your turn (3 min) - code break 7

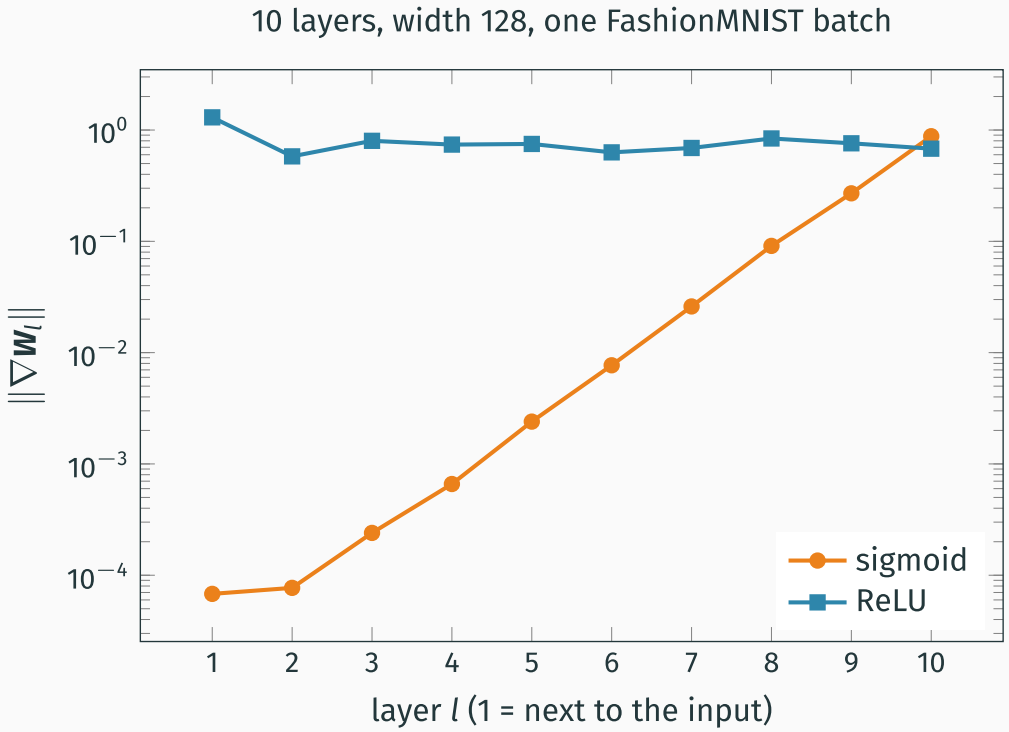
1. run it: the same loss three times - why does the slope grow?
2. put `w.grad = None` as the first line inside the loop. Now?

Vanishing gradients

Backprop multiplies one slope per layer.

- 20 layers, each slope 0.5: $0.5^{19} \approx 0.000002$
- sigmoid: slope at most 0.25 ⇒ the first layers get almost no signal
- ReLU: slope 1 (if active) ⇒ the signal passes

Fixes later: initialisation & normalisation (day 3), skip connections (day 4), gates (day 6)



Summary of the morning

🔑 Key takeaway

- Derivative = slope. Chain rule = multiply the small slopes along the chain
- MLP = layers of neurons; ReLU in between - without it the layers collapse into one
- In PyTorch: `nn.Sequential(nn.Linear(...), nn.ReLU(), ...)`
- Output layer + loss follow the task: MSE (regression), softmax + cross-entropy (classes)
- Backprop: forward and store, then backward with upstream × local
- PyTorch: `loss.backward()` does it for all weights; `zero_grad()` before each step
- Many layers with small slopes ⇒ vanishing gradients ⇒ prefer ReLU

Lab 2: MLPs & backprop

Run → Predict → Change → Explain

Lab 2

💻 `day2_mlp_backprop.ipynb`

part	what	min
0	warm-up: train your dragon in Python, slope, gradient descent	15
A	backprop step by step, checked by PyTorch	25
B	your first MLP in PyTorch: spirals, decision boundaries	30
C	PyTorch MLP on FashionMNIST: width, depth, activation	35
D	MLP on EEG band-power features (CHB-MIT)	25
E	⭐ optional: backprop by hand in NumPy	-

Every part: 1. run the example 2. predict 3. change 4. explain

Getting started

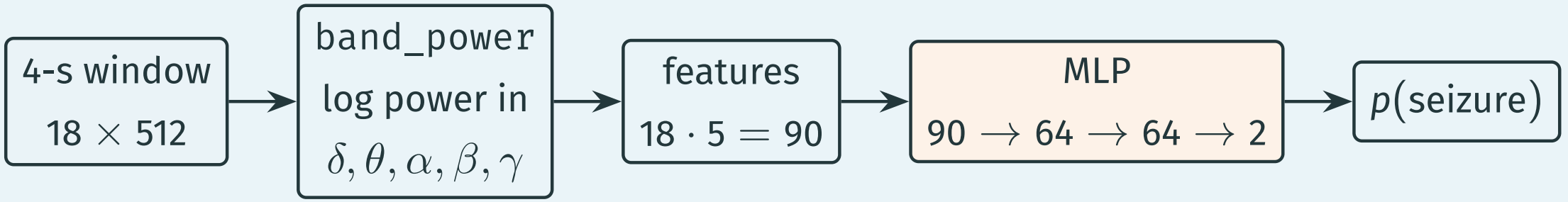
1. open the notebook in Colab
2. Runtime → Change runtime type → T4 GPU
3. work in pairs, run every cell from the top
4. ❌ in a check? Read the hint, then ask

Common errors and what they mean

error message (last line)	usual reason	fix
`NameError: name 'X' is not defined`	a cell above was not run	Runtime → Run before
`RuntimeError: mat1 and mat2 shapes cannot be multiplied (32x784 and 90x64)`	the layer expects a different input size	print `x.shape`, fix `nn.Linear(in, out)`
`RuntimeError: expected scalar type Float but found Double`	NumPy gives `float64`, PyTorch wants `float32`	`.float()` or `dtype=torch.float32`
`Expected all tensors to be on the same device`	model on GPU, data on CPU	`x = x.to(device)`
loss is `nan`	learning rate too large	smaller `lr`

MLPs on EEG features (Part D)

🧠 **From a 4-s window to p(seizure)**



- band power: how much energy each channel has in each frequency band (delta, theta, alpha, beta, gamma) - 18 × 5 = 90 numbers
- hand-crafted features + MLP: a classic and strong baseline for seizure detection
- the MLP sees a flat list of 90 numbers - it does not know which channels are neighbours ⇒ CNNs (day 4), GNNs (day 8)

❓ **Think about it**

Only 86 of 5081 windows are seizures. A model with 98 % accuracy - is it good?

Project: groups + 3 candidate papers

Wrap-up

Today

🔑 Key takeaway

- Derivative = slope; partial derivative = one knob at a time; chain rule = multiply the slopes
- An MLP stacks layers of neurons with ReLU in between - hidden layers learn new features
- Backpropagation = chain rule on the graph, from the loss back to every weight: upstream × local
- PyTorch: `requires_grad`, `loss.backward()`, `.grad`, `zero_grad()`

Day 3: where cross-entropy comes from, better optimisers (momentum, Adam), regularisation - and how to evaluate a seizure detector honestly.

Self-study

Watch / read

- 3Blue1Brown, Neural networks, chapters 3-4 (backpropagation) - 3blue1brown.com
- A. Karpathy, The spelled-out intro to neural networks and backpropagation: building micrograd (YouTube)
- D2L, sections 5.1-5.3 (MLPs, forward and backward propagation) - d2l.ai
- M. Nielsen, Neural Networks and Deep Learning, chapter 2

Questions?

Backup

Extra material - if we are faster than planned

Backup: what fits where

backup slide	min	fits after
Day 1 in 5 minutes (quiz)	5	start of the day
A derivative is a slope • derivative rules • symbols	10	"Gradient descent: roll downhill"
Hand exercises: chain rule + GD step, MSE table, one GD step, chain-rule practice	10-30	"The chain rule gives the MSE gradient"
Backprop through a tiny network by hand (+ solution)	15	"Try it: watch backprop in your browser"
Softmax by hand	5	"Softmax and cross-entropy with numbers"
Universal approximation with ReLUs	10	"Activation functions: ReLU and GELU"
Mini-batch forward pass • computation graph of an MLP	10	"Code break 6"
Backprop with matrices (5 slides) + from one neuron to whole layers (NumPy)	30	"Code break 6"
Autograd internals, TensorFlow / JAX	5	"Code break 6"
Checking gradients by turning the knob • choosing ε	10	"Code break 7"
Which EEG features matter? (saliency)	5	Lab Part D

Backup: day 1 in 5 minutes

👥 In pairs (5 min) - no slides, no notes

1. What does one neuron compute?
2. Why can a single perceptron not solve XOR?
3. What does the mean squared error measure?
4. What happens if the learning rate is too large?
5. Name the four lines of the PyTorch training loop.

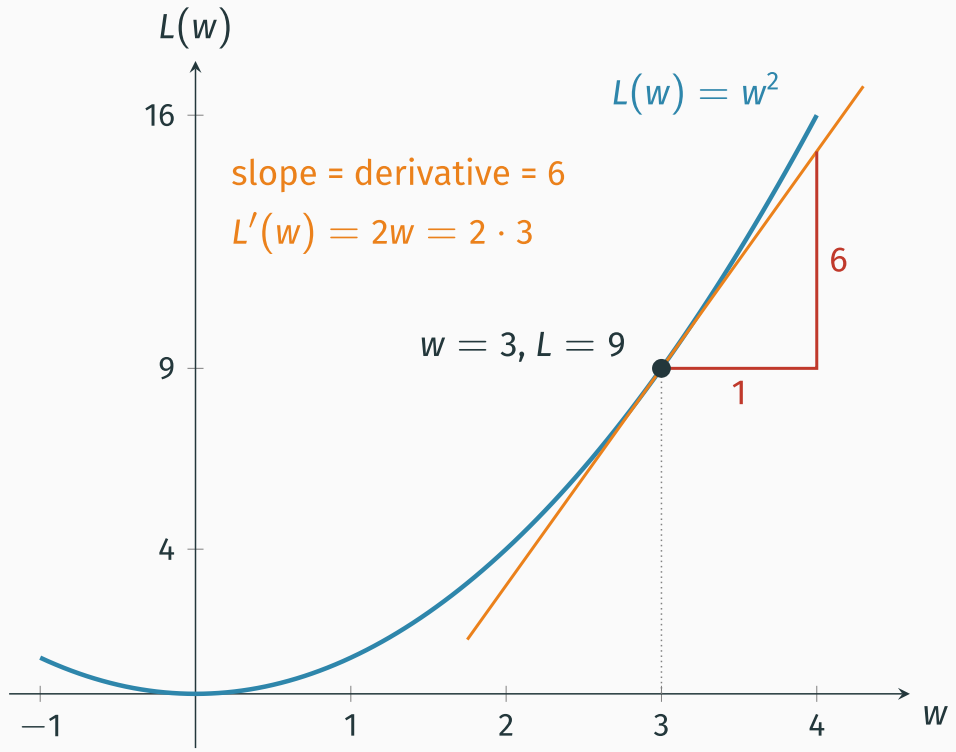
Backup: a derivative is a slope

Loss $L(w) = w^2$. How fast does L change if we turn the knob w a little?

w	$L(w)$
3.00	9.0000
3.01	9.0601
change	**0.0601**

slope $\approx \dfrac{0.0601}{0.01} = 6.01$

Rule: $\dfrac{dL}{dw} = 2w = 2 \cdot 3 = 6$ ✓



Backup: the derivative rules you need

rule	example	read aloud
power	$\frac{d}{dw} w^2 = 2w$	"the derivative of w squared is two w"
constant factor	$\frac{d}{dw} 3w = 3$	"a factor stays"
sum	$\frac{d}{dw}(w + 5) = 1$	"a constant disappears"
ReLU	$\frac{d}{dz} \max(0, z) = \begin{cases} 1 & z > 0 \\ 0 & z < 0 \end{cases}$	"one if positive, zero if negative"
chain rule	$\frac{d}{dw} f(g(w)) = f'(g(w)) \cdot g'(w)$	"outer slope times inner slope"

🔑 Key takeaway

That is all. Backpropagation = these rules + careful bookkeeping.

Backup: symbols and how to read them

symbol	read aloud	meaning
$\hat{y}$	"y hat"	prediction of the model
$e = \hat{y} - y$	"e equals y hat minus y"	error (residual)
$\sum_{i=1}^{n}$	"sum over i from 1 to n"	add up for every sample
$\frac{dL}{dw}$, $\frac{\partial L}{\partial w}$	"d L by d w", "partial L by partial w"	slope of L when only w moves
∇L	"nabla L", "the gradient of L"	list of all partial derivatives
η	"eta"	learning rate
$w \leftarrow w - \eta \nabla L$	"w becomes w minus eta times the gradient"	gradient-descent update
$\boldsymbol{W} \in \mathbb{R}^{4\times 3}$	"W is a real 4 by 3 matrix"	4 rows, 3 columns
φ, σ	"phi", "sigma"	activation function, sigmoid
$\odot$	"element-wise product"	multiply entry by entry

Backup: chain rule + gradient descent, worked example

	forward	small slope	slope of L (product)
$\hat{y} = w\,x + b$	$1 \cdot 2 + 0 = 2$	$\partial\hat{y}/\partial w = x = 2,\ \partial\hat{y}/\partial b = 1$	$\partial L/\partial w = -6 \cdot 2 = \mathbf{-12},\ \partial L/\partial b = -6 \cdot 1 = \mathbf{-6}$
$e = \hat{y} - y$	$2 - 5 = -3$	$\partial e/\partial \hat{y} = 1$	$\partial L/\partial \hat{y} = -6$
$L = e^2$	9	$\partial L/\partial e = 2e = -6$	-6

Gradient-descent step with $\eta = 0.1$:

$$w \leftarrow 1 - 0.1 \cdot (-12) = 2.2, \qquad b \leftarrow 0 - 0.1 \cdot (-6) = 0.6$$

New prediction: $\hat{y} = 2.2 \cdot 2 + 0.6 = 5.0 \Rightarrow L = 0$ 🎉

Backup: the MSE gradient on day-1 data

Day-1 data, start $w = b = 0$. Per sample: $\partial(e^2)/\partial w = 2e\,x$ and $\partial(e^2)/\partial b = 2e$. The MSE averages them:

x	y	$\hat{y}$	$e = \hat{y} - y$	e^2	$2e\,x$	$2e$
0	1	0	-1	1	0	-2
1	3	0	-3	9	-6	-6
2	5	0	-5	25	-20	-10
mean				$L = 11.67$	$\partial L/\partial w = -8.67$	$\partial L/\partial b = -6$

$$\frac{\partial L}{\partial w} = \frac{1}{n}\sum_{i=1}^{n} 2\,e_i\,x_i \qquad \frac{\partial L}{\partial b} = \frac{1}{n}\sum_{i=1}^{n} 2\,e_i$$

Step with $\eta = 0.1$: $w = 0.867$, $b = 0.6$ ⇒ new $L = 3.21$ (yesterday's pen-and-paper task - now solved)

Backup: one gradient-descent step by hand

One sample: $x = 1, y = 3$

Start: $w = 1, b = 0, \eta = 0.1$

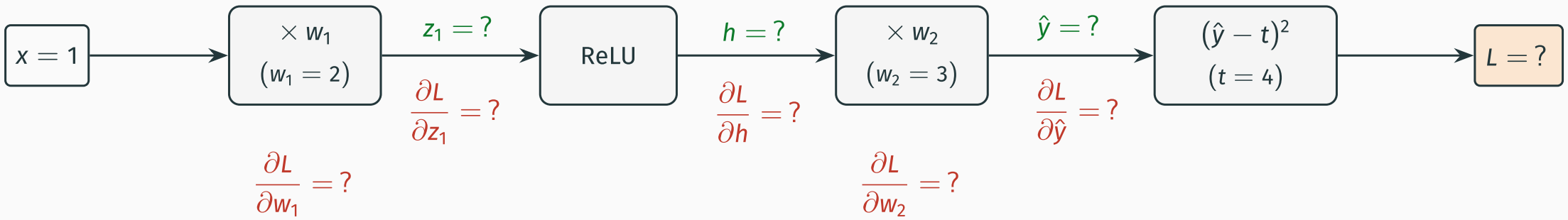
Model $\hat{y} = w\,x + b$, loss $L = (\hat{y} - y)^2$

Bonus: $x = 3, y = 0, w = 1, b = 1,$ $\eta = 0.01$

👥 Your turn (8 min, pen and paper)

1. forward: $\hat{y}$, e, L
2. small slopes: $\partial L/\partial e$, $\partial e/\partial \hat{y}$, $\partial \hat{y}/\partial w$, $\partial \hat{y}/\partial b$
3. multiply: $\partial L/\partial w$ and $\partial L/\partial b$
4. update w and b
5. new loss - did it go down?

Backup: backprop through a tiny network by hand

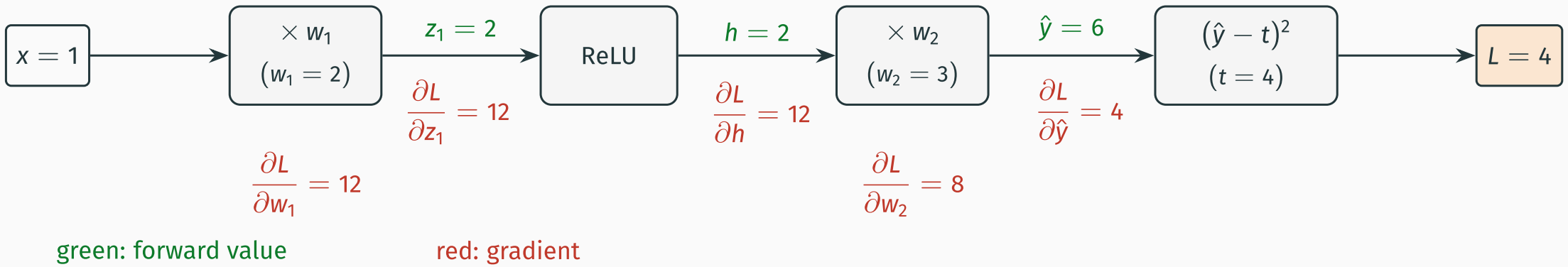


👥 Your turn (10 min, pen and paper)

Two layers, one neuron each, no bias: $x = 1$, $w_1 = 2$, $w_2 = 3$, target $t = 4$, loss $L = (\hat{y} - t)^2$

1. forward: z_1, $h = \mathrm{ReLU}(z_1)$, $\hat{y}$, L
2. backward from right to left: $\partial L/\partial \hat{y}$, $\partial L/\partial w_2$, $\partial L/\partial h$, $\partial L/\partial z_1$, $\partial L/\partial w_1$
3. Bonus: the same with $w_1 = -2$

Backup: solution - tiny network



- $\partial L/\partial w_2 = 8$: later layer - upstream × its input h
- $\partial L/\partial w_1 = 12$: earlier layer - re-uses $\partial L/\partial h$ from the later layer
- This is exactly what `loss.backward()` does - for millions of weights

Backup: more chain-rule practice

👥 Your turn (10 min)

1. $L(w) = (3w - 6)^2$. Compute dL/dw at $w = 1$. (Inner function: $3w - 6$.)
2. $L(w) = \mathrm{ReLU}(2w - 4)$. Compute dL/dw at $w = 1$ and at $w = 3$.
3. $L(w) = (w\,x - y)^2$ with $x = -1$, $y = 2$. Compute dL/dw at $w = 1$.
4. Check one of your answers by turning the knob: compute $L(w + 0.01)$.

Backup: softmax by hand

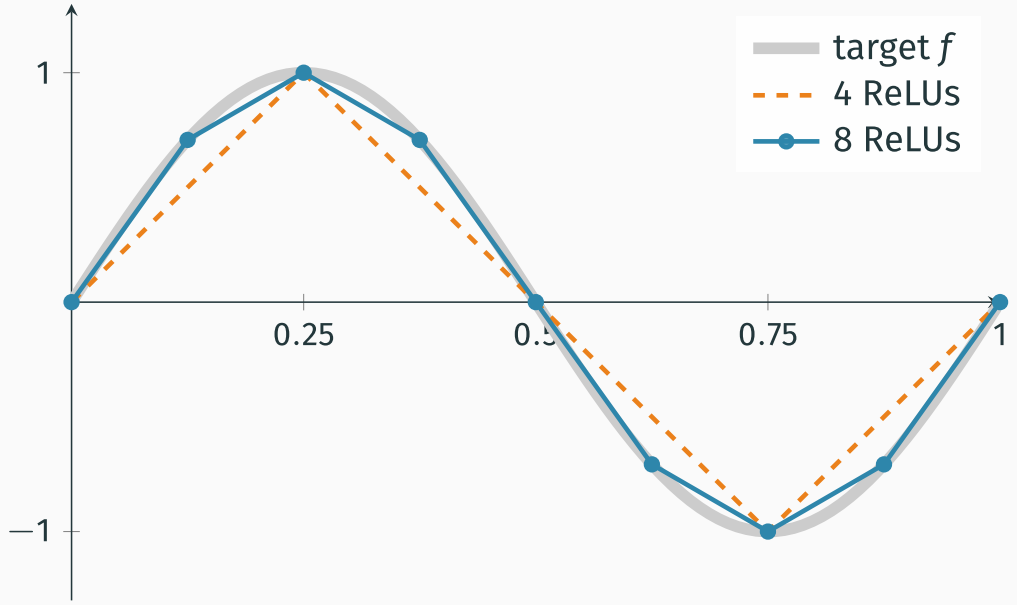
👥 Your turn (5 min)

Two classes (no seizure, seizure). Compute the softmax probabilities and the cross-entropy loss for the true class seizure:

1. logits $(0,\ 0)$
2. logits $(3,\ 0)$
3. logits $(0,\ 3)$

Hint: $e^0 = 1$, $e^3 \approx 20.09$

Backup: universal approximation with ReLUs

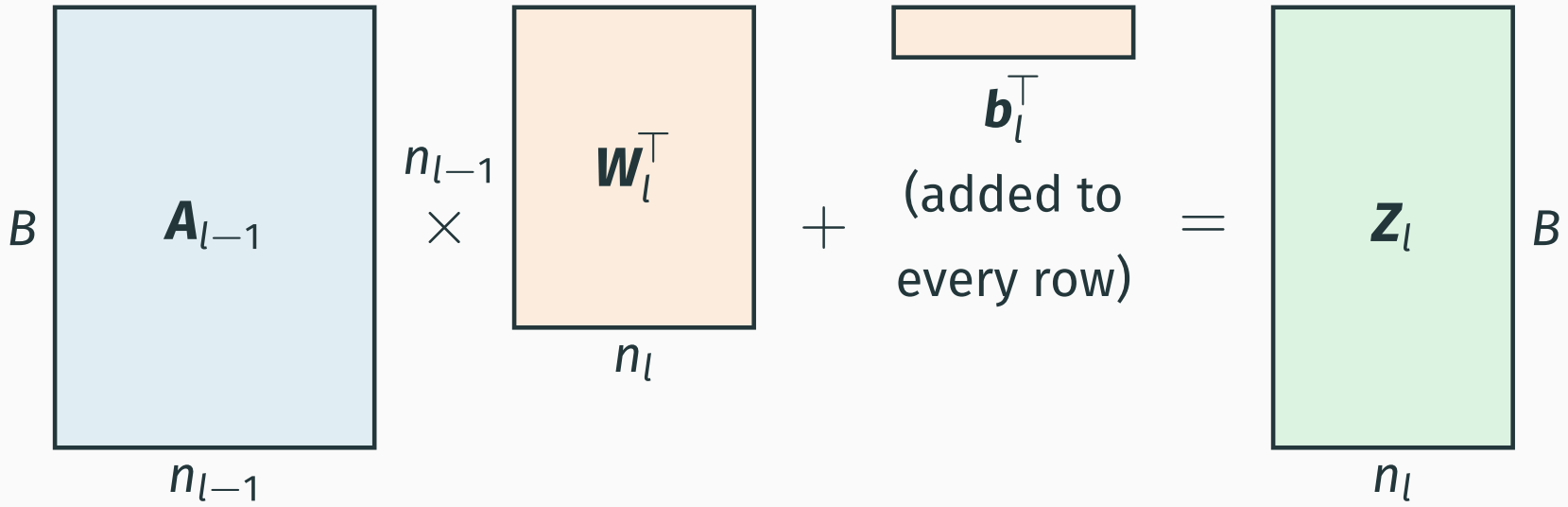


- One hidden ReLU unit $\mathrm{ReLU}(x - x_i)$ = a kink at x_i
- The output adds kinks: $\hat{f}(x) = f(0) + \sum_i c_i \,\mathrm{ReLU}(x - x_i)$
- More units ⇒ finer pieces ⇒ error → 0
- That is the universal approximation theorem in 1D

Example (4 units): $\hat{f}(x) = 4\,\mathrm{ReLU}(x) - 8\,\mathrm{ReLU}(x - \frac{1}{4}) + 8\,\mathrm{ReLU}(x - \frac{3}{4})$

Cybenko 1989; Hornik 1991; Nielsen, Neural Networks and Deep Learning, ch. 4

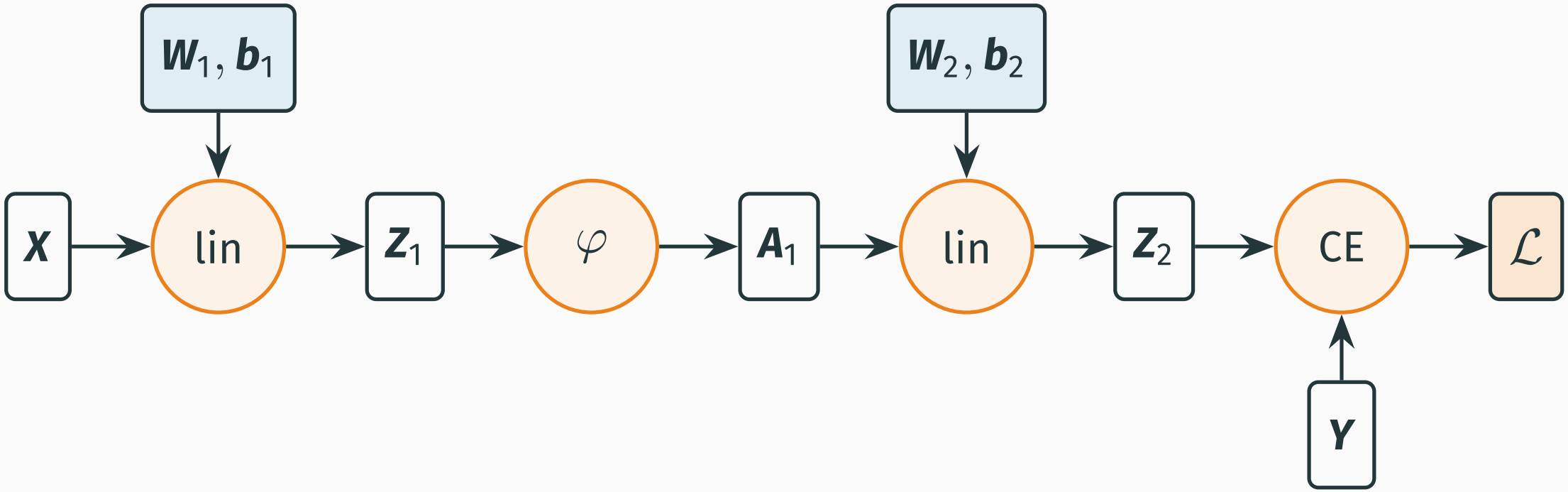
Backup: forward pass for a mini-batch



$$A_0 = X, \qquad Z_l = A_{l-1} W_l^\top + b_l, \qquad A_l = \varphi(Z_l)$$

- one sample per row: row i of Z_l belongs to sample i
- exactly `nn.Linear`: `weight` has shape (out, in), so we multiply with its transpose
- one matrix product per layer ⇒ fast on GPUs; no Python loops over samples

Backup: the computation graph of an MLP



- orange circles = operations, boxes = tensors, blue = parameters
- forward: left to right; backward: right to left (next backup slides)

❓ Think about it

Which tensors must be kept in memory after the forward pass so that we can compute the gradients later?

Backup: backprop with matrices - the setting

$$\boldsymbol{Z}_1 = \boldsymbol{X}\boldsymbol{W}_1^\top + \boldsymbol{b}_1$$
$$\boldsymbol{A}_1 = \varphi(\boldsymbol{Z}_1)$$
$$\boldsymbol{Z}_2 = \boldsymbol{A}_1\boldsymbol{W}_2^\top + \boldsymbol{b}_2$$
$$\boldsymbol{P} = \mathrm{softmax}(\boldsymbol{Z}_2)\ \text{(per row)}$$
$$L = -\frac{1}{B}\sum_{i=1}^{B}\sum_{k=1}^{K} Y_{ik}\log P_{ik}$$

tensor	shape
$\boldsymbol{X}$	$B \times d$
$\boldsymbol{W}_1, \boldsymbol{b}_1$	$H \times d, H$
$\boldsymbol{Z}_1, \boldsymbol{A}_1$	$B \times H$
$\boldsymbol{W}_2, \boldsymbol{b}_2$	$K \times H, K$
$\boldsymbol{Z}_2, \boldsymbol{P}, \boldsymbol{Y}$ (one-hot)	$B \times K$

Notation: error signal $\boldsymbol{\Delta}_l = \partial L / \partial \boldsymbol{Z}_l$ (same shape as $\boldsymbol{Z}_l$)

Backup: step 1 - the output error

One sample, true class c:

$$\ell = -\log p_c = -z_c + \log \sum_j e^{z_j} \quad \Rightarrow \quad \frac{\partial \ell}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_j e^{z_j}} = p_k - y_k$$

Mean over the batch:

$$\boldsymbol{\Delta}_2 = \frac{\partial L}{\partial \boldsymbol{Z}_2} = \frac{1}{B}(\boldsymbol{P} - \boldsymbol{Y}) \qquad (B \times K)$$

- "prediction minus target": large when confidently wrong, ≈ 0 when right
- MSE with identity output gives the same form: $\frac{1}{B}(\boldsymbol{Z}_2 - \boldsymbol{Y})$ (with the factor $\frac{1}{2}$ in the loss)

Backup: steps 2 and 3 - weights, then the hidden layer

Step 2 - weights of layer 2 ("error × input", summed over the batch):

$$\nabla \boldsymbol{W}_2 = \boldsymbol{\Delta}_2^\top \boldsymbol{A}_1 \quad (K \times H), \qquad \nabla \boldsymbol{b}_2 = \text{column sums of } \boldsymbol{\Delta}_2$$

Step 3 - back into the hidden layer (through the transposed weights, then the ReLU gate):

$$\frac{\partial L}{\partial \boldsymbol{A}_1} = \boldsymbol{\Delta}_2 \boldsymbol{W}_2 \quad (B \times H), \qquad \boldsymbol{\Delta}_1 = (\boldsymbol{\Delta}_2 \boldsymbol{W}_2) \odot \varphi'(\boldsymbol{Z}_1)$$

$$\nabla \boldsymbol{W}_1 = \boldsymbol{\Delta}_1^\top \boldsymbol{X}, \qquad \nabla \boldsymbol{b}_1 = \text{column sums of } \boldsymbol{\Delta}_1$$

- shape check: $(K \times B)(B \times H) = K \times H$ ✓ - the shape of $\boldsymbol{W}_2$
- ReLU: $\varphi'(\boldsymbol{Z}_1) = \mathbf{1}[\boldsymbol{Z}_1 > 0]$ - only active units pass the error

Backup: backprop for L layers

Algorithm (one mini-batch)

forward: $\boldsymbol{A}_0 = \boldsymbol{X}$; for $l = 1, \ldots, L$: $\boldsymbol{Z}_l = \boldsymbol{A}_{l-1}\boldsymbol{W}_l^\top + \boldsymbol{b}_l$, $\boldsymbol{A}_l = \varphi(\boldsymbol{Z}_l)$ (store!)

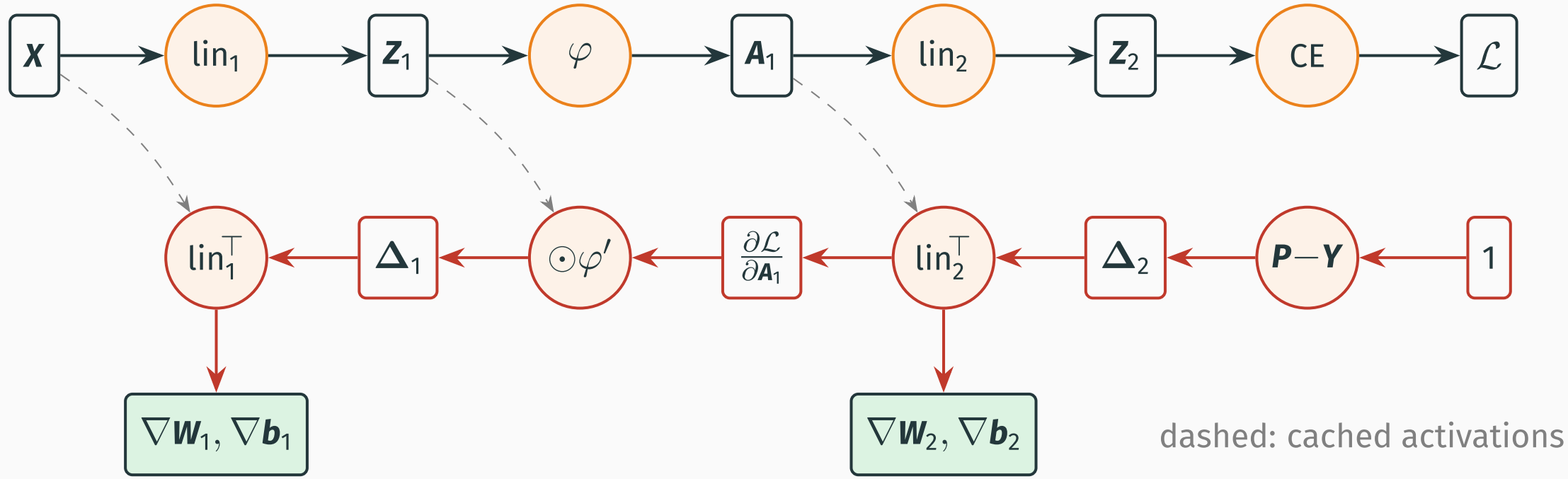
output error: $\boldsymbol{\Delta}_L = \frac{1}{B}(\boldsymbol{P} - \boldsymbol{Y})$

backward: for $l = L, \ldots, 1$:
$\nabla \boldsymbol{W}_l = \boldsymbol{\Delta}_l^\top \boldsymbol{A}_{l-1}$, $\nabla \boldsymbol{b}_l$ = column sums of $\boldsymbol{\Delta}_l$
if $l > 1$: $\boldsymbol{\Delta}_{l-1} = (\boldsymbol{\Delta}_l \boldsymbol{W}_l) \odot \varphi'(\boldsymbol{Z}_{l-1})$

update: $\boldsymbol{W}_l \leftarrow \boldsymbol{W}_l - \eta \nabla \boldsymbol{W}_l$, $\boldsymbol{b}_l \leftarrow \boldsymbol{b}_l - \eta \nabla \boldsymbol{b}_l$

- $\boldsymbol{\Delta}_l$ is computed once and re-used for $\nabla \boldsymbol{W}_l$ and for $\boldsymbol{\Delta}_{l-1}$ - this is why backprop is cheap
- 3 matrix products per layer: backward ≈ 2 × forward
- price: memory for all stored activations

Backup: the backward pass is the reversed graph



- every forward machine has a backward machine; the arrows point the other way
- the backward machines need the stored forward values (dashed) ⇒ training needs memory for all activations

Backup: from one neuron to whole layers (NumPy)

The same rules, for all neurons and all samples at once - written with matrices:

```
# forward (store Z1, A1, P)
Z1 = X @ W1.T + b1          # (B, H)   weighted sums, layer 1
A1 = np.maximum(0, Z1)      # (B, H)   ReLU
P  = softmax(A1 @ W2.T + b2)  # (B, K) probabilities

# backward: upstream x local, layer by layer
D2  = (P - Y) / B           # (B, K)   slope at the output: prediction - target
dW2 = D2.T @ A1             # (K, H)   "error x input" of layer 2
db2 = D2.sum(axis=0)        # (K,)
D1  = (D2 @ W2) * (Z1 > 0)  # (B, H)   back through W2, then the ReLU gate
dW1 = D1.T @ X              # (H, d)   "error x input" of layer 1
db1 = D1.sum(axis=0)        # (H,)
```

You do **not** need to derive this. In the lab it is the optional Part E - for the curious.

Backup: checking gradients by turning the knob

If you write gradients yourself - check them against "turning the knob":

$$\frac{\partial L}{\partial w} \approx \frac{L(w+\varepsilon) - L(w-\varepsilon)}{2\varepsilon}, \qquad \varepsilon \approx 10^{-6}$$

- costs 2 forward passes per weight - only for testing on a tiny network
- they should agree to about 6-7 digits (use `float64`)
- in the lab: optional Part E

Backup: autograd internals - and other frameworks

```
x = torch.randn(4, 3)
W = torch.randn(2, 3, requires_grad=True)
L = torch.relu(x @ W.T).sum()
print(L.grad_fn)
# <SumBackward0 ...>
print(L.grad_fn.next_functions)
# ((<ReluBackward0 ...>, 0),)
```

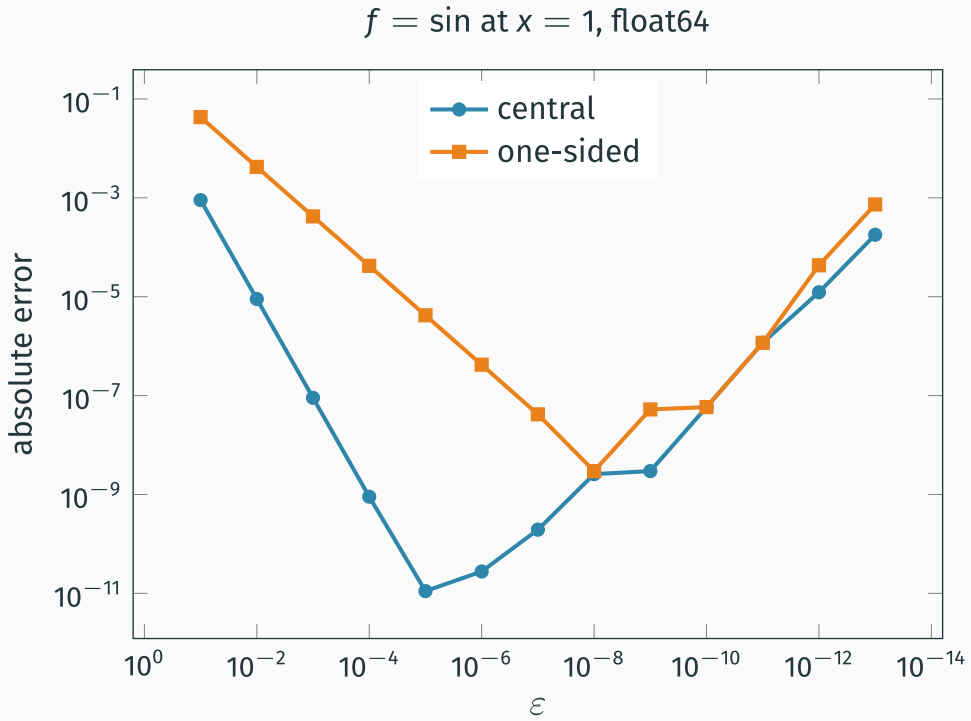
PyTorch builds the graph while your code runs ("define-by-run"): `if` and loops just work.

```
import tensorflow as tf
w, b = tf.Variable(2.0), tf.Variable(-1.0)
with tf.GradientTape() as tape:
    L = (tf.nn.relu(w * 3.0 + b) - 4.0) ** 2
print(tape.gradient(L, [w, b]))   # 6.0, 2.0
```

- TensorFlow: record on a "tape", replay in reverse
- JAX: `jax.grad(f)` returns the gradient function
- the maths is the same everywhere - we use PyTorch

Paszke et al., NeurIPS 2019 (PyTorch)

Backup: gradient check - choosing ε



$f = \sin$ at $x = 1$, float64, error of the numerical slope:

- ε too large: the "straight line" between the two points is too rough
- ε too small: rounding errors of the computer
- turning to both sides (central) is much more accurate than to one side
- compare: $\frac{\|g_{\text{backprop}} - g_{\text{numerical}}\|}{\|g_{\text{backprop}}\| + \|g_{\text{numerical}}\|} \lesssim 10^{-7}$

Backup: which EEG features matter?

🧠 Gradients with respect to the input

Backprop also gives $\partial p(\text{seizure})/\partial x$: how sensitive is the prediction to each of the 90 band-power features?

```
x = feats[i:i+1].clone().requires_grad_(True)   # one window, track the input
p = model(x).softmax(dim=1)[0, 1]               # p(seizure)
p.backward()
saliency = x.grad.abs().reshape(18, 5)          # channel x frequency band
```

- a saliency map over channels × bands
- local and noisy - a first look, not an explanation (more on day 8)

Simonyan, Vedaldi & Zisserman, ICLR Workshop 2014